

ΕΠΕΞΕΡΓΑΣΙΑ ΦΥΣΙΚΗΣ ΓΛΩΣΣΑΣ

Εργασία 3

Η εργασία στοχεύει στην εξοικείωση με την επισημείωση ακολουθιών (μία από τις πιο βασικές και χρήσιμες εργασίες) και τη χρήση προ-εκπαιδευμένων γλωσσικών μοντέλων (η πιο καινοτόμα τεχνική για την πραγματοποίηση ποικίλων εργασιών) στην περιοχή της Επεξεργασίας Φυσικής Γλώσσας.

Επισημείωση Ακολουθιών (Sequence Labeling)

Οι πιο βασικές περιπτώσεις επισημείωσης ακολουθιών είναι οι εξής:

- **Επισημείωση μέρους-του-Λόγου (part-of-Speech tagging):** Σε κάθε token μιας πρότασης εισόδου αποδίδεται ένα tag που δείχνει το μέρος-του-Λόγου (π.χ. ουσιαστικό, επίθετο, ρήμα κλπ.) στο οποίο αντιστοιχεί.
- **Αναγνώριση ονοματικών οντοτήτων (named-entity recognition):** Σε κάθε token μιας πρότασης εισόδου αποδίδεται ένα tag που δείχνει αν αποτελεί μέρος μιας ονοματικής οντότητας καθώς και τον τύπο της ονοματικής οντότητας (π.χ. όνομα προσώπου, τοποθεσίας, οργανισμού κλπ).
- **Αναγνώριση ορίων φράσεων (text chunking):** Σε κάθε token μιας πρότασης εισόδου αποδίδεται ένα tag που δείχνει αν αποτελεί μέρος μιας φράσης καθώς και τον τύπο της φράσης (π.χ. ονοματική, ρηματική κλπ).

Για παράδειγμα, η παρακάτω πρόταση από το dataset CoNLL003¹:

EU rejects German call to boycott British lamb.

έχει επισημειωθεί με POS tags και Chunk tags και NER-tags ως εξής:

Tokens	POS tags	Chunk tags	NER tags
EU	NNP	B-NP	B-ORG
rejects	VBZ	B-VP	O
German	JJ	B-NP	B-MISC
call	NN	I-NP	O
to	TO	B-VP	O
boycott	VB	I-VP	O
British	JJ	B-NP	B-MISC
lamb	NN	I-NP	O
.	.	O	O

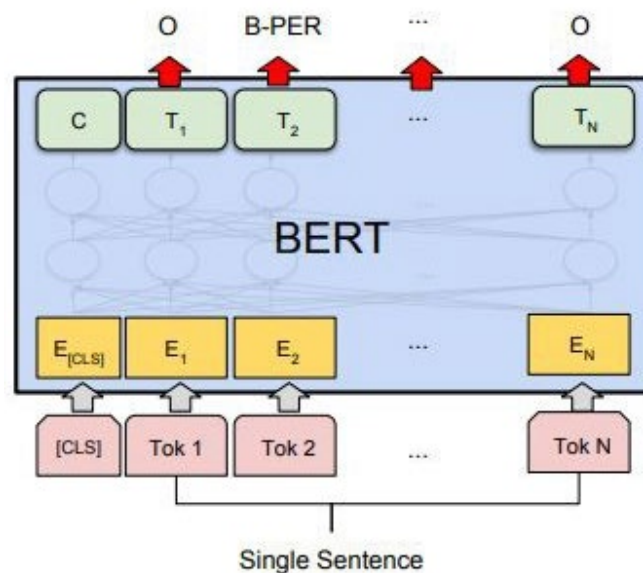
- **POS tagging:** υπάρχουν 2 ουσιαστικά ("call" και "lamb"), 2 επίθετα ("German", "British"), 1 κύριο όνομα ("EU"), 1 ρήμα ("boycott"), 1 ρήμα τρίτου προσώπου ("rejects"), 1 πρόθεση ("to") και ένα σημείο στίξης (".")
- **Text chunking:** υπάρχουν 3 ονοματικές φράσεις ("EU", "German call", "British lamb"), 2 ρηματικές φράσεις ("rejects", "to boycott") ενώ ένα token (.) δεν αποτελεί μέρος κάποιας φράσης.
- **NER:** υπάρχουν 3 ονοματικές οντότητες, 2 τύπου MISC ("German", "British") και 1 τύπου ORG ("EU") ενώ όλα τα υπόλοιπα tokens δεν αποτελούν μέρος ονοματικής οντότητας.

¹ <https://www.kaggle.com/datasets/alaakhaled/conll003-englishversion?select=train.txt>

Το CoNLL003 dataset περιέχει 20.744 προτάσεις που έχουν επισημειωθεί παρόμοια με το παραπάνω παράδειγμα. Από αυτές, 14.041 προτάσεις περιλαμβάνονται στο training set, 3.250 προτάσεις στο validation set και 3.453 προτάσεις στο test set.

Προ-εκπαιδευμένα Γλωσσικά Μοντέλα (Pre-trained Language Models)

Ένα προ-εκπαιδευμένο γλωσσικό μοντέλο έχει εκπαιδευτεί με χρήση μεγάλων συλλογών κειμένων και μπορεί να παρέχει contextualized embeddings για κάθε token μιας πρότασης εισόδου. Τα embeddings αυτά είναι πυκνά διανύσματα (χωρίς πολλά 0) που συμπυκνώνουν τις ιδιότητες του κάθε token ενώ εξαρτώνται από τα συμφραζόμενα. Τα τελευταία χρόνια έχουν γίνει διαθέσιμα πολλά τέτοια προ-εκπαιδευμένα γλωσσικά μοντέλα (π.χ. BERT, RoBERTa) τα περισσότερα από τα οποία βασίζονται στην αρχιτεκτονική του Transformer². Ένα τέτοιο μοντέλο μπορεί να χρησιμοποιηθεί και για την επισημείωση ακολουθιών όπως φαίνεται στο παρακάτω σχήμα:



Για να χρησιμοποιηθεί ένα τέτοιο μοντέλο απαιτείται η χρήση ενός ειδικού tokenizer που έχει χρησιμοποιηθεί και στην φάση εκπαίδευσής του. Επίσης, υπάρχει ένα ανώτατο όριο μήκους της ακολουθίας εισόδου. Ο tokenizer εισάγει 2 ειδικά tokens [CLS] και [SEP] στην αρχή και το τέλος της ακολουθίας αντίστοιχα. Για κάθε token εισόδου παράγεται ένα embedding (διάνυσμα) στην έξοδο το οποίο μπορεί να χρησιμοποιηθεί ως αναπαράσταση του token και με ένα επιπλέον επίπεδο ταξινόμησης για να γίνει η εκτίμηση της πιθανότητας να ανήκει το token σε κάθε δυνατό tag (για κάθε tag δημιουργείται μία έξοδος και ένα fully-connected layer συνδέει την κάθε τιμή των embeddings με καθεμία έξοδο). Στη φάση της εκπαίδευσης ενός τέτοιου μοντέλου για εφαρμογές επισημείωσης ακολουθίας μαθαίνονται εξ αρχής οι παράμετροι που αφορούν το επίπεδο ταξινόμησης και τροποποιούνται οι παράμετροι του προ-εκπαιδευμένου μοντέλου (fine-tuning) ώστε να ταιριάζει καλύτερα με τις ανάγκες της εφαρμογής.

Εκτός από encoder-only μοντέλα, όπως το BERT, υπάρχουν και decoder-only προ-εκπαιδευμένα μοντέλα, όπως τα GPT, Llama, Mistral. Σε αυτήν την περίπτωση δίνουμε στο μοντέλο μια οδηγία (prompt) και την προς ανάλυση πρόταση, και του ζητάμε να επιστρέψει το αποτέλεσμα σε δομημένη

² [https://en.wikipedia.org/wiki/Transformer_\(machine_learning_model\)](https://en.wikipedia.org/wiki/Transformer_(machine_learning_model))

μορφή. Το μοντέλο είναι ικανό να ακολουθήσει οδηγίες και να δημιουργήσει την απάντηση, χωρίς να έχει εκπαιδευτεί ειδικά για το συγκεκριμένο task.

Ερωτήματα

Δίνεται κώδικας (ner-bert.py) που υλοποιεί στην python ένα named-entity recognizer με χρήση του προ-εκπαιδευμένου μοντέλου BERT. Πιο συγκεκριμένα, χρησιμοποιείται η έκδοση **bert-base-uncased**³ που έχει περιορισμό εισόδου 512 subword tokens και παρέχει embeddings διάστασης 768 για το κάθε token. Επίσης, χρησιμοποιείται ένα classification layer στην έξοδο του μοντέλου για κάθε token το οποίο ταξινομεί το token σε ένα από τα διαθέσιμα tags⁴. Για την εκπαίδευση και τον έλεγχο της επίδοσης χρησιμοποιούνται τα training, validation, test sets του **CoNLL003** dataset⁵.

1. Εκτελέστε τον κώδικα σε 3 επαναλήψεις και αναφέρετε την μέση τιμή και την τυπική απόκλιση που επιτυγχάνεται στο test set για τις παρακάτω μετρικές:
 - a. Token-level micro-average accuracy
 - b. Token-level macro-average accuracy
 - c. Entity-level micro-average F1
 - d. Entity-level macro-average F1
 - e. Training time cost
2. Εξηγήστε ποια είναι η διαφορά μεταξύ token-level και entity-level μετρικών αξιολόγησης. Ποιο από τα δύο θεωρείτε ότι ταιριάζει καλύτερα με το NER task;
3. Για μία συγκεκριμένη εκπαίδευση του μοντέλου, επιλέξτε μία πρόταση από το test set με τουλάχιστον 10 tokens στην οποία το μοντέλο αποτυγχάνει να βρει τα σωστά tags για κάποια tokens. Δείξτε αναλυτικά σε ποια tokens αποδίδονται τα σωστά tags και σε ποια tokens γίνεται λάθος. Επιπλέον, εισάγετε ως είσοδο στο τελικό μοντέλο μία νέα πρόταση (π.χ. από μια online εφημερίδα της επιλογής σας) με τουλάχιστον 10 tokens στην οποία να εμφανίζονται κάποιες ονοματικές οντότητες και δείξτε σε ποια tokens το μοντέλο κάνει σωστή και σε ποια λάθος πρόβλεψη.
4. Εξηγήστε ποιος είναι ακριβώς ο ρόλος της συνάρτησης align_label. Για ποιο λόγο θέτει τα ids κάποιων tokens στην τιμή -100;
5. Τροποποιήστε τον αρχικό κώδικα και επαναλάβετε το ερώτημα 1 έχοντας παγώσει τα βάρη που αφορούν το προ-εκπαιδευμένο γλωσσικό μοντέλο BERT, οπότε στη φάση της εκπαίδευσης θα καθοριστούν μόνο τα βάρη του επιπέδου ταξινόμησης. Αναφέρετε ποιες αλλαγές έγιναν στον κώδικα. Αναφέρετε πόσες παράμετροι του μοντέλου έχουν γίνει freeze και πόσες όχι. Πώς συγκρίνονται τα αποτελέσματα με αυτά του ερωτήματος 1;
6. Τροποποιήστε τον αρχικό κώδικα ώστε αντί για αναγνώριση ονοματικών οντοτήτων να πραγματοποιείται επισημείωση μέρους-του-Λόγου (POS tagging). Αναφέρετε ποιες αλλαγές έγιναν στον κώδικα. Επαναλάβετε τα ερωτήματα 1⁶ και 3 για τον τροποποιημένο κώδικα.

³ <https://huggingface.co/bert-base-uncased>

⁴ https://huggingface.co/docs/transformers/model_doc/bert#transformers.BertForTokenClassification

⁵ <https://www.kaggle.com/datasets/alaakhalel/conll003-englishversion>

⁶ Αγνοήστε τις entity-level μετρικές για το POS tagging

7. Τροποποιήστε τον αρχικό κώδικα ώστε αντί για αναγνώριση ονοματικών οντοτήτων να πραγματοποιείται αναγνώριση ορίων φράσεων (text chunking). Αναφέρετε ποιες αλλαγές έγιναν στον κώδικα. Επαναλάβετε τα ερωτήματα 1 και 3 για τον τροποποιημένο κώδικα.
8. Τροποποιήστε τον αρχικό κώδικα για αναγνώριση ονοματικών οντοτήτων ώστε να χρησιμοποιείται το προ-εκπαιδευμένο μοντέλο **roberta-base**⁷ (όπως και ο αντίστοιχος tokenizer) και επαναλάβετε το ερώτημα 1 για τον τροποποιημένο κώδικα. Αναφέρετε ποιες αλλαγές έγιναν στον κώδικα. Συγκρίνετε τα αποτελέσματα που προέκυψαν με αυτά του ερωτήματος 1.
9. Με βάση το **Groq**⁸ (ή άλλη πλατφόρμα πρόσβασης σε LLMs της επιλογής σας) χρησιμοποιήστε το μοντέλο **Llama-3.1-8B**⁹ για να αναγνωρίσετε τις ονοματικές οντότητες στις πρώτες 200 προτάσεις του test set με zero-shot prompting. Σε κάθε κλήση του μοντέλου να δίνεται μία μόνο πρόταση προς ανάλυση με τις κατάλληλες οδηγίες. Αναφέρετε το prompt που χρησιμοποιήθηκε. Πώς συγκρίνονται τα αποτελέσματα με αυτά του ερωτήματος 1 και 8 (για τις 200 πρώτες προτάσεις του test set);
10. Επαναλάβετε το προηγούμενο ερώτημα αυτή την φορά χρησιμοποιώντας το μοντέλο **Llama-3.3-70B**¹⁰. Συγκρίνετε τα αποτελέσματα με αυτά των ερωτημάτων 1, 8 και 9 και σχολιάστε την επίδοση και το χρονικό κόστος των μοντέλων. Πόσο ανταγωνιστικά είναι τα decoder-only μοντέλα σε σύγκριση με τα encoder-only μοντέλα;

Οδηγίες:

Η εκτέλεση του κώδικα χωρίς τη χρήση επιταχυντή (GPU) απαιτεί πολλές ώρες ανά εποχή. Συστήνεται η χρήση του **Kaggle**¹¹ ή **Colab**¹² για να εκτελέσετε τον κώδικα (με δωρεάν χρήση GPU εντός κάποιων ορίων). Για παράδειγμα, με χρήση accelerator GPU-P100 απαιτούνται περίπου 15 λεπτά ανά εποχή.

Η πλατφόρμα **Groq** επιβάλλει όρια κλήσεων και tokens ανά λεπτό¹³. Για να μπορέσετε να εκτελέσετε τις κλήσεις στα LLMs που χρειάζονται για τις 200 προτάσεις θα πρέπει να εισάγετε χρονική καθυστέρηση μεταξύ διαδοχικών κλήσεων. Φροντίστε να αποθηκεύετε τα αποτελέσματα μετά από κάθε κλήση.

Η εργασία είναι **ατομική**.

Τι πρέπει να υποβάλλετε:

- Μία **αναφορά (PDF)** με τις απαντήσεις στα ερωτήματα της εργασίας καθώς και τις συγκεκριμένες αλλαγές που πραγματοποιήθηκαν σε τμήματα του κώδικα όπου ζητείται. Στο τέλος της αναφοράς, απαντήστε στα παρακάτω:
 - Υπάρχουν ερωτήματα που δεν καταφέρατε να απαντήσετε ή δεν είστε σίγουροι για την ορθότητα των απαντήσεών σας;
 - Υπάρχει κάποιο πειραματικό αποτέλεσμα που σας φάνηκε απροσδόκητο;

⁷ <https://huggingface.co/roberta-base>

⁸ <https://console.groq.com/home>

⁹ Στο Groq φαίνεται ως **llama-3.1-8b-instant**

¹⁰ Στο Groq φαίνεται ως **llama-3.3-70b-versatile**

¹¹ <https://www.kaggle.com/code>

¹² <https://research.google.com/colaboratory/faq.html>

¹³ <https://console.groq.com/docs/rate-limits>